

CNN1D-Autoencoder para Detecção de Anomalias em Turbina

Migração de metodologia do AutoML, normalização restrita ao período operacional e o candidato EXP15b

Pipeline `cnn1d_ae` — projeto TesteMLCab (Cabiunas/Petrobras)

Agosto de 2026

Resumo

Este relatório documenta a evolução do candidato de referência do CNN1D-Autoencoder (CNN1D-AE) da série EXP13 para o novo candidato **EXP15b**, motivada pela investigação de dois episódios reais não detectados pelo candidato anterior (2026-03-25 e 2026-04-14). A investigação separou as causas: o episódio de 2026-03-25 não era falha do modelo (o autoencoder detectou, mas um portão de segurança bloqueou a detecção por uma margem pequena demais para o mecanismo de resgate existente); o de 2026-04-14 era uma falha real – uma deriva lenta de temperatura (+115°C em ~24h) invisível ao modelo porque as estatísticas de normalização eram contaminadas por ~42% de dados de treino em período de parada/partida, comprimindo o sinal padronizado de desvios reais dentro da operação. A correção (`NORMALIZE_ON_STATE_ONLY`, restringindo as estatísticas de normalização ao período operacional) mais uma recalibração de threshold guiada por regrid offline resultou no candidato **EXP15b**: cobertura de alarme (`hit_rate`) de **77,5% (31/40)**, superando o candidato anterior (75,0%, 30/40), com falso alerta de **0,30%** – ainda bem abaixo do melhor candidato AutoML de referência (0,35%). Documentamos também a origem híbrida da metodologia atual: boa parte dos mecanismos de pós-processamento do CNN1D-AE (segundo critério da máscara operacional, portão de rampa refinado, portão de volatilidade) foi *herdada* de uma série de experimentos AutoML (EXP7–EXP10) que investigou e reduziu o falso alerta de um candidato `ocsvm` em ~82% relativo antes de essas correções serem portadas de volta para o CNN1D-AE.

Sumário

1	Introdução e contexto	2
2	Migração de metodologia: do AutoML para o CNN1D-AE	2
2.1	Infraestrutura de avaliação compartilhada	2
2.2	Features multiescala e de textura (herdado do AutoML EXP7)	3
2.3	Máscara operacional: segundo critério (herdado do AutoML EXP10)	3
2.4	Portão de rampa: originou no CNN1D-AE, refinado no AutoML (EXP10b)	3
2.5	Portão de volatilidade: nasceu no AutoML, portado para o CNN1D-AE (EXP10c)	4
2.6	Resumo da migração	4
3	Metodologia	4
3.1	Dados	4
3.2	Pré-processamento e janelas	5
3.3	Arquitetura e treino do autoencoder	5
3.4	Threshold e mapeamento sequência→ponto	6
3.5	Camada de pós-processamento	6
3.6	Contribuição nova: normalização restrita ao período operacional	8

4	Resultados	8
4.1	Evolução do candidato	8
4.2	Estudo de caso: o episódio de 2026-04-14	9
4.3	Classificação dos alarmes por qualidade de detecção	10
4.3.1	Antecedência de detecção, caso a caso	11
4.4	Variação entre sementes de retreino (seed-sweep)	13
4.5	Séries completas com anomalias marcadas	13
5	Discussão	14
5.1	Tentativa de mitigação de variância rejeitada	14
5.2	Limitações	14
5.3	Pendências	15
6	Conclusão	15
A	Tasks ClearML referenciadas	15

1 Introdução e contexto

O projeto `TesteMLCab` desenvolve detectores de anomalia para sensores de uma turbina a gás da Cabiunas (Petrobras), com foco inicial em temperatura de gás de escape (`TC382_03_A/T5_AVG_A`) e nos 10 canais de vibração de mancal associados. Duas famílias de pipeline coexistem no repositório e compartilham infraestrutura de avaliação:

- **AutoML** (`src/cnn1d_ae/automl_pipeline.py`) — modelos ponto-a-ponto (`dense/iforest/ocsvm`) sobre um vetor de features por instante, sem janelamento temporal explícito no modelo. Testados numa grade ampla de hiperparâmetros (percentil de `threshold`, `debounce`) por sensor/grupo.
- **CNN1D-AE** (`src/cnn1d_ae/pipeline.py`) — o objeto deste relatório: um autoencoder convolucional 1-D que reconstrói janelas deslizantes multivariadas, buscado por KerasTuner e avaliado pelo erro de reconstrução (MAE).

As duas pipelines evoluíram em paralelo e, mais importante para este relatório, **trocaram descobertas entre si**: mecanismos de pós-processamento nascidos numa pipeline foram portados para a outra sempre que uma investigação de falso positivo/falso negativo revelava uma causa física que a outra pipeline também sofria. A Seção 2 documenta essa migração em detalhe. A Seção 3 descreve a metodologia atual completa (dados, janelas, split, arquitetura), incluindo a normalização restrita ao período operacional – a contribuição nova documentada aqui. A Seção 4 apresenta os resultados comparativos, incluindo o estudo de caso do episódio 2026-04-14 e a classificação dos 40 alarmes do período de avaliação por qualidade de detecção. A Seção 5 discute limitações, uma tentativa de mitigação rejeitada (`THRESH_MODE=target_rate`) e pendências.

2 Migração de metodologia: do AutoML para o CNN1D-AE

Nem toda melhoria no CNN1D-AE nasceu dentro dele. A série de experimentos AutoML EXP5–EXP11 (`docs/analise_automl_exp*.md`) funcionou como um laboratório mais barato de iterar – os modelos ponto-a-ponto treinam em segundos a minutos contra horas do CNN1D-AE – e várias descobertas de lá foram portadas de volta. Esta seção documenta essa migração na ordem em que aconteceu.

2.1 Infraestrutura de avaliação compartilhada

Antes de qualquer migração de *mecanismo*, as duas pipelines já compartilhavam a mesma régua de avaliação, definida em `src/cnn1d_ae/scoring.py` e usada por ambas:

- **Split out-of-sample (OOS) temporal:** o modelo (e, no CNN1D-AE, a normalização e o threshold) só enxerga dados anteriores a `OOS_SPLIT_DATE`; a avaliação de `hit_rate/normal_alert_rate` só considera alarmes e pontos posteriores ao corte. A escolha do corte não é arbitrária – EXP5 (AutoML) documentou um achado negativo em que mover o corte OOS sem checar a distribuição de alarmes destruiu o poder estatístico da avaliação (a amostra de alarmes na fatia de teste caiu de 70 para 4–8).
- **eval_alarm_hit_rate:** para cada alarme real do catálogo, verifica se existe algum ponto marcado como anômalo numa janela de $\pm \text{EXCLUDE_MINUTES_AROUND_ALARM}$ (24h) ao redor do timestamp do alarme.
- **compute_composite_score:** combina `hit_rate` e `normal_alert_rate` (falso alerta) num único escore, penalizando falso alerta quadraticamente – usado para ranquear candidatos/trials.
- **Seed-sweep:** re-treina a mesma arquitetura vencedora com N sementes extras e reporta a variação de `hit_rate/normal_alert_rate`. Nasceu no AutoML (EXP5, `AUTOML_SEED_SWEEP_N`, motivado por uma variância de $\sim 27\text{pp}$ observada num autoencoder `dense` anterior) e foi portado para o CNN1D-AE, onde revelou o mesmo problema em escala menor (Seção 4.4).

Essa infraestrutura compartilhada é o que torna a comparação AutoML $\leftrightarrow$ CNN1D-AE do restante deste relatório legítima: ambos são avaliados pela mesma função sobre o mesmo conjunto de 40 alarmes OOS de TC382_03_A/T5_AVG_A.

2.2 Features multiescala e de textura (herdado do AutoML EXP7)

O EXP7 (AutoML) introduziu features derivadas em múltiplas escalas de tempo – média/desvio móvel e tendência (`roll_med/roll_std/trend`) nas janelas de 6 min, 1 h, 4 h e 24 h (`DERIVED_ROLLING_WINDOWS = [12, 120, 480, 2880]` amostras a 30s) – e, para janelas ≥ 60 amostras, features de “textura” do sinal (*kurtosis*, *skewness*, *crest factor* = pico/RMS), comuns em *condition monitoring* de vibração para capturar um sinal ficando mais “impulsivo” antes de uma falha de mancal. O ganho reportado no AutoML foi limpo e sem custo adicional de falso alerta. O mesmo bloco de código (`_build_derived_features`, `preprocess.py`) e os mesmos parâmetros de janela são usados, sem alteração, no CNN1D-AE – é a mesma implementação compartilhada entre as duas pipelines, não uma reimplementação.

2.3 Máscara operacional: segundo critério (herdado do AutoML EXP10)

O EXP10 (AutoML) fragmentou a série de falso alerta em episódios contínuos e cruzou cada um com o dado bruto e o catálogo completo de 47 tags de alarme (não só os 2 sensores avaliados). Achado: 65,3% do falso alerta positivo (OOS) vinha de um único desligamento real de 3,5 dias em que TC382_03_A caía de $\sim 478^\circ\text{C}$ para $\sim 28\text{--}32^\circ\text{C}$ enquanto `RUNNING_A` (referência da máscara operacional) permanecia em $\sim 0,96\text{--}1,0$ o tempo todo – a máscara nunca reconhecia esse período como “desligado”. A correção: `build_operational_state` (`scoring.py`) ganhou um segundo critério de “off”, baseado no próprio sensor-alvo do grupo caindo abaixo de um piso físico (`secondary_series/secondary_off_abs_threshold`, ou `OFF_TARGET_ABS_THRESHOLD` na config), unido por OR ao critério de `OPERATIONAL_REF_SENSOR` antes da classificação `off_curto/off_longo/transiente`. Redução de FP no AutoML: 1,94% $\rightarrow$ 0,67%, `hit_rate` idêntico (92,5%). Essa mesma função é usada pelo CNN1D-AE (config atual: `OFF_TARGET_ABS_THRESHOLD=150,0^\circ\text{C}`).

2.4 Portão de rampa: originou no CNN1D-AE, refinado no AutoML (EXP10b)

Diferente dos demais mecanismos desta seção, o portão de rampa (`ENABLE_LOAD_GATE/apply_load_gate`) **já existia no CNN1D-AE antes** e nunca havia sido portado para o AutoML. O EXP10b portou-o, mas os parâmetros *default* do CNN1D-AE (*halflife*=120 min/janela=360 min) custavam até 6 detecções preditivas genuínas no AutoML – uma rampa de falha real e uma manobra de carga legítima têm a mesma assinatura de taxa de variação numa janela longa. Uma janela mais curta (*halflife*=15 min/janela=30 min) + `ramp_max=100°C/h`, encontrados por simulação offline sobre os dados reais do AutoML, preservaram 29/29 detecções preditivas com FP caindo de 0,67% para 0,48%. Esse achado de janela-curta-preserva-preditivos informou a recalibração do próprio portão de rampa do CNN1D-AE em rodadas posteriores do EXP13 (ver `docs/analise_cnn1dae_exp13.md`) – uma migração de *ida e volta* entre as duas pipelines.

2.5 Portão de volatilidade: nascido no AutoML, portado para o CNN1D-AE (EXP10c)

O portão de rampa reage a variação de *nível*, mas o padrão de falso alerta residual do EXP10b era diferente: o desvio-padrão dos canais de vibração subia e **permanecia** elevado por 1–2h – não um pico de subida, mas uma volatilidade persistente. A primeira tentativa (alimentar o portão de rampa existente com um índice de volatilidade de vibração no lugar do proxy de temperatura, mantendo o mecanismo de *taxa de variação*) falhou: reduzia o FP de 0,480% para apenas 0,477% – taxa de variação não distingue o início de uma rampa real do início de uma escalada de falha, o mesmo problema do portão de rampa original, agora por métrica errada em vez de janela errada. A segunda tentativa funcionou e é um mecanismo genuinamente **novo**, sem equivalente prévio em nenhuma das duas pipelines: `compute_volatility_index/apply_volatility_gate` (`scoring.py`) – desvio-padrão móvel causal de cada canal de vibração, reduzido pela média entre canais, bloqueando quando o **nível** (não a taxa) ultrapassa um `threshold`. Simulação offline encontrou o ponto de custo zero: FP caindo mais 27% relativo (0,48%→0,35%), preservando as 29/29 detecções preditivas. Esse mecanismo – inteiramente uma descoberta do AutoML – foi portado para o CNN1D-AE no EXP13 (commit 542d203, “port dos gates EXP10 pro CNN1D-AE”) e é hoje um dos quatro portões da camada de pós-processamento descrita na Seção 3.3.

2.6 Resumo da migração

Mecanismo	Origem	Direção da migração
Split OOS temporal, <code>eval_alarm_hit_rate</code> , <code>composite_score</code>	Compartilhado desde o início	Infraestrutura comum (<code>scoring.py</code>)
Seed-sweep	AutoML (EXP5)	AutoML → CNN1D-AE
Features multiescala + textura	AutoML (EXP7)	AutoML → CNN1D-AE (mesma implementação)
2º critério da máscara operacional	AutoML (EXP10)	AutoML → CNN1D-AE
Portão de rampa (parâmetros refinados)	CNN1D-AE (original) → AutoML (EXP10b)	CNN1D-AE → AutoML → CNN1D-AE (recalibração)
Portão de volatilidade	AutoML (EXP10c)	AutoML → CNN1D-AE (EXP13)
Bloqueio gradual (<i>gate-escape</i>)	CNN1D-AE (EXP13)	Original, sem equivalente no AutoML
Normalização restrita ao período operacional	CNN1D-AE (este relatório, EXP15b)	Original, sem equivalente no AutoML

Tabela 1: Origem e direção de migração de cada mecanismo da pipeline atual.

3 Metodologia

3.1 Dados

- **Fonte:** `sensores_full_2024_2026_30s.csv` – 12 canais a 30 s de resolução: TC382_03_A (alvo), T5_AVG_A (referência de carga) e 10 canais de vibração de mancal (`TV_35{1..5}{X,Y}_A`).
- **Período:** `DATA_START_DATE=2024-07-01` até o fim da série disponível (~abril de 2026).
- **Alarmes:** `alarmes_selecionados_turbina_a.csv` – catálogo de alarmes do SCADA. A avaliação usa só eventos de ativação (`Status` iniciando em `ACT`, excluindo pares de recuperação `INACT`) dos sensores TC382_03_A/ T5_AVG_A, no período OOS: **40 alarmes**.
- **Split OOS:** `OOS_SPLIT_DATE=2025-07-01` – treino (modelo + normalização + threshold) só vê dados anteriores; avaliação só considera alarmes/pontos posteriores.

3.2 Pré-processamento e janelas

O pipeline segue sete fases, documentadas em detalhe em `docs/preprocessamento_entrada_modelo.md` e resumidas na Figura 2:

1. **Carregamento e normalização temporal** – parsing de timestamp, conversão para UTC naïve.
2. **Construção do dataframe do grupo** – alinhamento dos 12 sensores no mesmo índice, interpolação de lacunas curtas (≤ 3 amostras), máscara de gaps longos (OR entre sensores).
3. **Máscara de exclusão de alarme** – janela bilateral de ± 24 h ao redor de cada alarme excluída do treino (`EXCLUDE_MINUTES_AROUND_ALARM=1440`).
4. **Clipping de outliers** – modo quantile (percentis 0,1%/99,9%), limites calculados só sobre dados de treino.
5. **Normalização treino-apanas** – z-score (média/desvio), estatísticas calculadas exclusivamente sobre dados sem alarme e sem gap longo. **Contribuição nova deste relatório:** restrição adicional ao período operacional, Seção 3.6.
6. **Sequências deslizantes** – janelas de `TIME_STEPS=60` amostras (30 min a 30 s/amostra) com `STRIDE=15` (7,5 min entre janelas consecutivas) – tensor ($N, 60, 12$).
7. **Split treino/validação** – temporal, 90%/10% (`VAL_FRAC=0,1`), sem embaralhar (evita vazamento entre janelas sobrepostas).

O princípio central, preservado em todas as versões da pipeline: as estatísticas de pré-processamento (clipping, normalização, threshold) são calculadas exclusivamente sobre dados normais, mas a **inferência roda sobre a série completa**, incluindo períodos anômalos – é essa assimetria que faz uma anomalia elevar o erro de reconstrução.

3.3 Arquitetura e treino do autoencoder

O CNN1D-AE (`src/cnn1d_ae/model.py`) é buscado por KerasTuner (`RandomSearch`, `MAX_TRIALS=40`) sobre o espaço:

Hiperparâmetro	Espaço de busca
filtros da 1ª/2ª camada conv.	{8, 16, 32, 64}
tamanho de kernel 1ª/2ª camada	{3, 5, 7, 9}
<i>dropout</i>	[0, 0, 0, 4], passo 0,1
taxa de aprendizado	{ 10^{-4} , 3×10^{-4} , 10^{-3} , 3×10^{-3} }
<i>stride</i> 1ª/2ª camada conv.	{1, 2}

Arquitetura (encoder–decoder simétrico, ativação ReLU, saída linear):

$$\text{Conv1D}(f_1, k_1, s_1) \rightarrow \text{Dropout} \rightarrow \text{Conv1D}(f_2, k_2, s_2) \rightarrow \text{Conv1DTranspose}(f_2, k_2, s_2) \\ \rightarrow \text{Dropout} \rightarrow \text{Conv1DTranspose}(f_1, k_1, s_1) \rightarrow \text{Conv1DTranspose}(12, 3)$$

Otimizador Adam, perda MSE, `EarlyStopping (patience=5, restore_best_weights=True)` + `ReduceLROnPlateau`. Tanto o candidato anterior (task 14) quanto o EXP15b convergiram para a **mesma arquitetura vencedora** (seed principal 42, mesmo espaço de busca): $f_1=64$, $f_2=32$, $k_1=k_2=7$, $\text{dropout}=0$, $lr=3 \times 10^{-4}$, $s_1=s_2=1$ – a diferença de resultado entre os dois candidatos vem inteiramente da normalização de entrada e da recalibração do threshold, não de uma arquitetura diferente.

O erro de reconstrução é o MAE médio no eixo temporal, por sequência e por canal (`reconstruction_mae_per`, quando o grupo define um `target_sensor` (aqui, TC382_03_A), o threshold e a detecção usam só o MAE desse canal, não a média entre os 12 canais do grupo – os demais canais entram como *features* de contexto para o autoencoder, não como alvo de detecção.

3.4 Threshold e mapeamento sequência→ponto

O threshold usa o modo `robust_mad`: mediana + $K \times 1,4826 \times \text{MAD}$ do MAE de treino – robusto a caudas longas que fariam `mean_std` produzir um limiar inatingível. Sequências com MAE acima do threshold são mapeadas para pontos via `POINT_RULE=k_of_window (POINT_WINDOW=2, POINT_MIN_COUNT=1)`: um ponto é anômalo se pelo menos 1 das últimas 2 sequências que o cobrem cruzou o threshold.

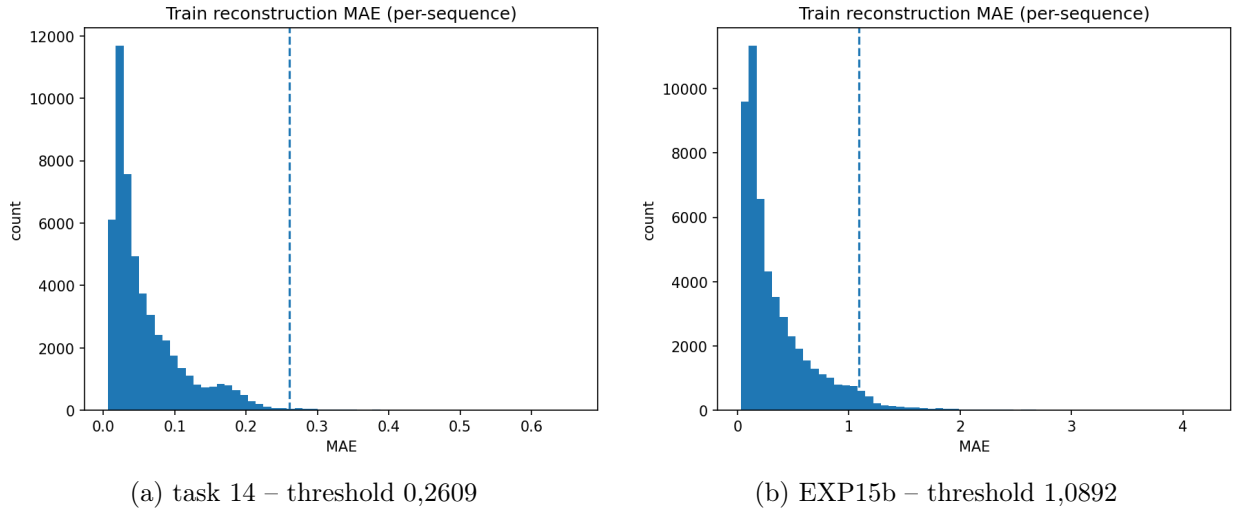


Figura 1: Distribuição do MAE de treino (canal-alvo) e o threshold `robust_mad` resultante, para cada candidato. A cauda longa à direita é exatamente o padrão que motiva usar mediana+MAD em vez de média+desvio-padrão – poucos pontos de MAE alto no treino já inflam `mean_std` a um patamar que a série inteira nunca cruza.

3.5 Camada de pós-processamento

Antes de virar um ponto anômalo reportado, a detecção passa por quatro filtros – todos os quatro atuam sobre a saída do modelo já treinado, sem alterar pesos nem o threshold em si (Figura 2):

1. **Máscara operacional** (Seção 2.3) – suprime detecção fora do estado `on`.
2. **Portão de rampa** (`LOAD_GATE_SENSOR=T5_AVG_A`, `RAMP_MAX=50°C/h`) – suprime durante manobra de carga legítima.

3. **Portão de volatilidade** (VOLATILITY_GATE_THRESHOLD=0,1745, janela de 30 min) – suprime durante vibração elevada persistente e fisicamente plausível.
4. **Bloqueio gradual** (GATE_ESCAPE_MULTIPLIER=1,5, Seção 2) – um ponto cujo MAE bruto ultrapassa $\text{threshold} \times 1,5$ escapa do bloqueio de qualquer portão, mesmo sem saber qual bloqueou nem por quê. Nasceu do episódio 2026-01-29 do EXP13 (dois portões bloqueando simultaneamente uma detecção real) e é reaproveitado sem alteração no EXP15b.

Pipeline CNN1D-AE — do dado bruto ao ponto anômalo avaliado

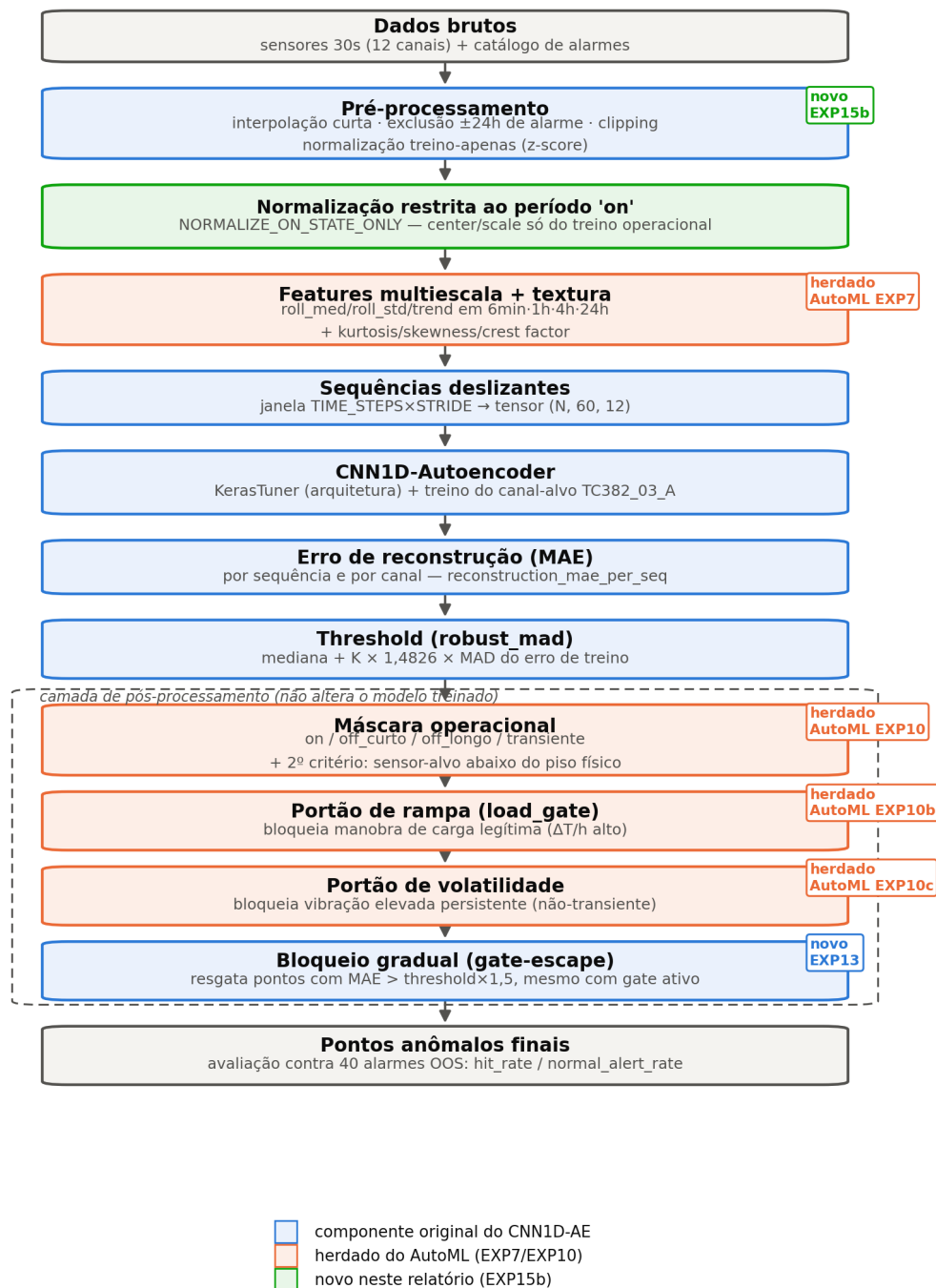


Figura 2: Esquema completo da pipeline, do dado bruto ao ponto anômalo avaliado. Azul: componentes originais do CNN1D-AE. Laranja: mecanismos herdados do AutoML (Seção 2). Verde: contribuição nova deste relatório.

3.6 Contribuição nova: normalização restrita ao período operacional

A investigação que motiva este relatório partiu de dois episódios reais não detectados pelo candidato de referência anterior (task 14, Seção 4). O episódio de **2026-04-14** revelou uma falha estrutural na normalização: `normalize_train_only` (`preprocess.py`) sempre calculou `center/scale` sobre *todo* o `df_normal_fit` (dados de treino, excluindo só janelas de alarme e gaps longos) – **sem** excluir os períodos de parada/partida da turbina. Como $\sim 42\%$ do treino tem `TC382_03_A` $< 100^\circ\text{C}$ (turbina desligada), o desvio-padrão usado para o z-score é inflado por essa mistura bimodal: $51,1^\circ\text{C}$ considerando só o período operando versus $323,0^\circ\text{C}$ misturado com parada/partida – quase $6,3\times$ maior.

O efeito prático: um desvio real de $+115^\circ\text{C}$ *dentro* da operação normal (o episódio de 2026-04-14) produz um z-score de apenas 1,22 com as estatísticas contaminadas, contra 2,22 se calculado só sobre o período operacional – quase metade da magnitude estatística disponível para o autoencoder aprender a distinguir esse desvio do ruído normal.

Implementação (`NORMALIZE_ON_STATE_ONLY`, `config.py` + `preprocess.py` + `pipeline.py`): `normalize_train_only` recebe um `stats_mask` opcional – quando fornecido, `center/scale` são calculados só sobre as linhas de `df_normal_fit` onde `operational_state=='on'`. Duas decisões de design deliberadas:

- **Não filtra as linhas de treino em si** – as sequências de treino continuam contíguas no tempo (sem remover janelas off/partida e recolar os pedaços), evitando descontinuidades artificiais nas sequências que o autoencoder aprende a reconstruir. Só as *estatísticas* usadas para normalizar mudam.
- **Cálculo do estado operacional antecipado** – no `run_one_group`, o cálculo de `state` (`build_operational_state`) foi movido para antes de `normalize_train_only` (antes rodava só depois, para mascarar detecções). Comportamento idêntico ao anterior quando a flag está desligada (`False` por default) – nenhum experimento existente muda de resultado.

Esse ganho de sinal tem um efeito colateral que a Seção 5 discute: os períodos off/partida continuam nas sequências de treino, agora normalizados com um desvio-padrão menor – um valor de $\sim 33^\circ\text{C}$ (desligado) vira um z-score de $\approx -12,7$ em vez de $\approx -1,1$, forçando o autoencoder a reconstruir transições artificialmente amplificadas. Isso exigiu uma recalibração de threshold não-trivial (não um simples reescalonamento linear) para chegar ao candidato final.

4 Resultados

4.1 Evolução do candidato

A Tabela 2 e a Figura 3 mostram a evolução completa, do melhor candidato AutoML (referência externa) até o EXP15b, todos avaliados na mesma régua (40 alarmes OOS de `TC382_03_A/T5_AVG_A`).

Candidato	hit_rate	FP	Observação
AutoML EXP10c (referência)	92,5% (37/40)	0,35%	ocsvm, 3 portões, ver Seção 2
CNN1D-AE task 13	60,0% (24/40)	0,17%	THRESH_STD_K=7,0, sem gate-escape
CNN1D-AE task 14 (anterior)	75,0% (30/40)	0,22%	+ gate-escape (resgata 2026-01-29)
CNN1D-AE EXP15b (novo)	77,5% (31/40)	0,30%	+ normalização on-state + K=4,5

Tabela 2: Evolução do candidato de referência do CNN1D-AE, com o melhor candidato AutoML como teto de referência.

Evolução do candidato: AutoML EXP10c → CNN1D-AE EXP15b

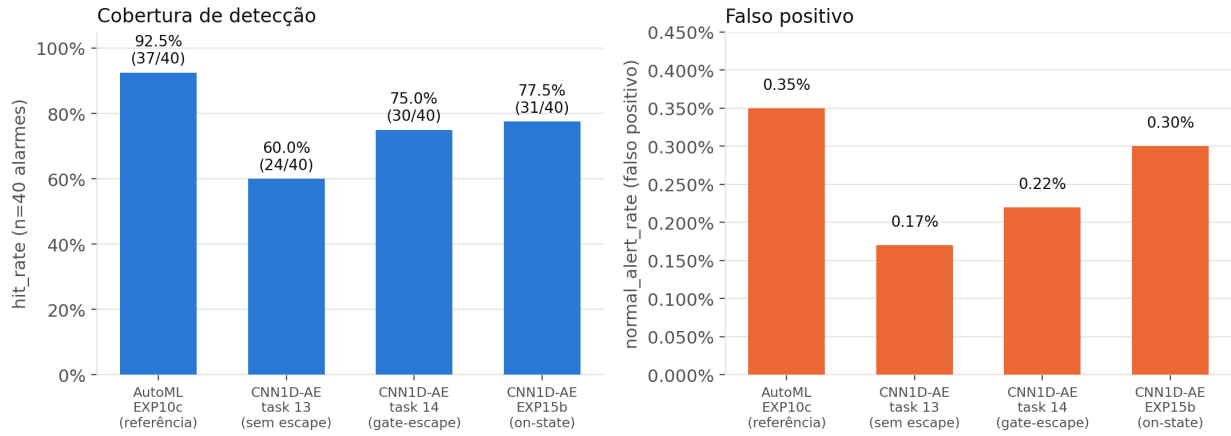


Figura 3: **hit_rate** e falso positivo (**normal_alert_rate**) ao longo da evolução do candidato. O EXP15b supera o candidato anterior (task 14) nos dois eixos simultaneamente... exceto pelo FP, que sobe ~37% relativo (0,22%→0,30%) – ainda assim bem abaixo do teto do AutoML (0,35%).

4.2 Estudo de caso: o episódio de 2026-04-14

A Figura 4 mostra o mecanismo do resgate diretamente nos dados. No painel superior, a temperatura bruta de TC382_03_A sobe suavemente de ~677°C (13/04, 12h) para ~793°C (14/04, 12h) ao longo de ~24h – sem descontinuidade local que uma janela de 30 min pudesse capturar como “forma” anômala. No painel inferior, o erro de reconstrução do canal-alvo: a curva do candidato anterior (task 14, azul) mal reage – fica sempre abaixo do seu próprio threshold (0,26) mesmo com a temperatura 115°C acima do início da janela. A curva do EXP15b (laranja) cruza claramente o seu threshold (1,09) já na madrugada de 14/04 e se mantém acima na maior parte da janela, incluindo nos três horários de alarme reais (HI às 00h12, HI às 10h07, HIHI às 12h01, marcados em vermelho).

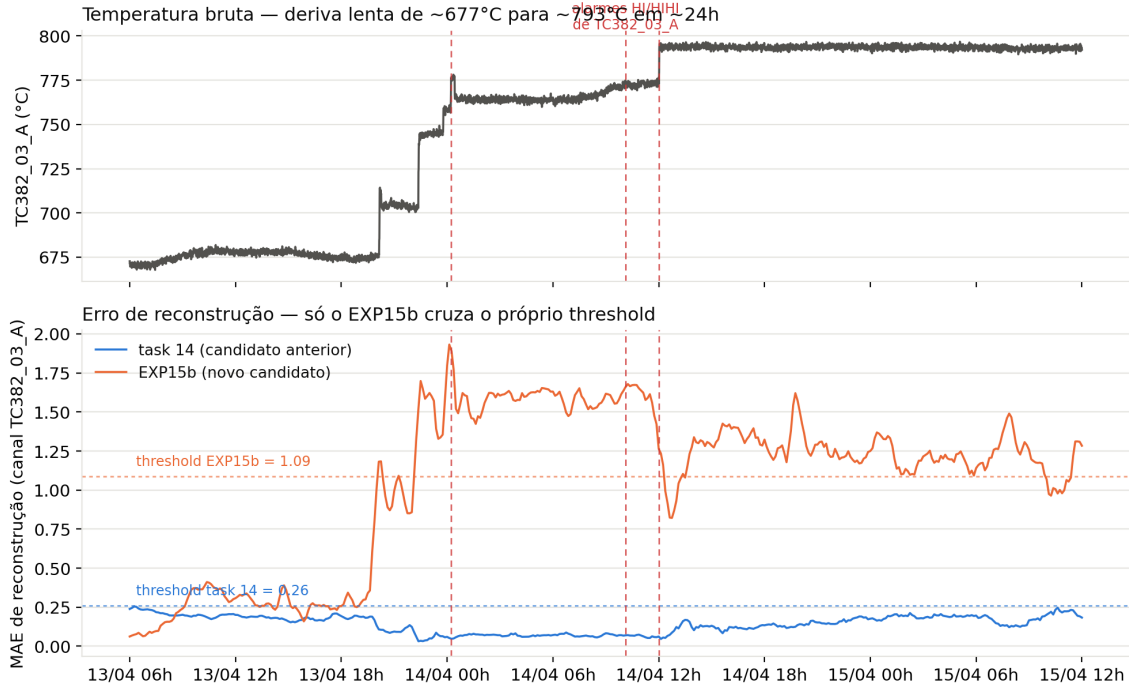


Figura 4: Episódio 2026-04-14: temperatura bruta (topo) e erro de reconstrução dos dois candidatos (base), com os respectivos thresholds. Note que os dois eixos de MAE não são diretamente comparáveis em escala absoluta – a normalização de entrada mudou entre os dois candidatos – o que importa é a posição de cada curva *relativa ao seu próprio* threshold.

Confirmação no nível de ponto (após máscara operacional e portões): 138 e 161 pontos anômalos nas janelas de $\pm 24h$ dos dois principais alarmes do dia, mesmo com `load_gate/volatility_gate` ativos na região – resgatados pelo *gate-escape*, o mesmo mecanismo que já havia resgatado o episódio 2026-01-29 no candidato anterior.

4.3 Classificação dos alarmes por qualidade de detecção

Além da métrica agregada `hit_rate` (que só pergunta “existe algum ponto anômalo na janela de $\pm 24h$?”), classificamos cada um dos 40 alarmes pelo tempo de antecedência da primeira detecção dentro dessa janela: **preditivo** (primeira detecção antes do alarme, até 20h de antecedência), **suspeito** (antecedência $> 20h$ – padrão já documentado de artefato: quando a detecção está perto do teto da janela de avaliação, é mais provável ser coincidência da janela ser larga do que sinal precursor real), **reativo** (detecção no instante do alarme ou depois) e **sem detecção**.

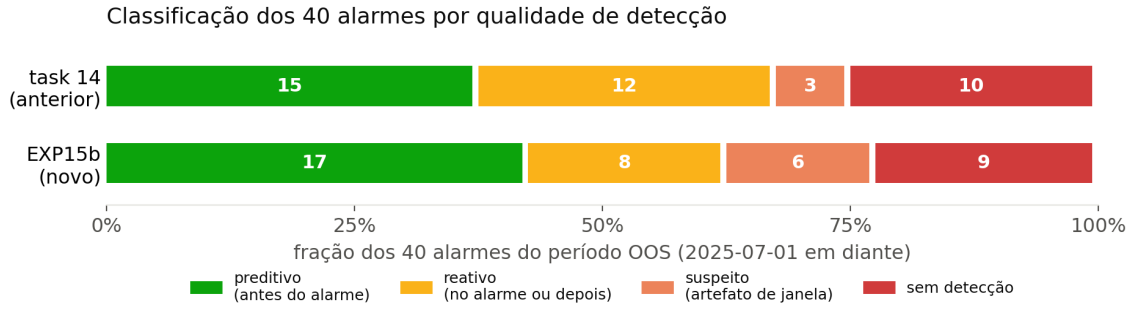


Figura 5: Classificação automática dos 40 alarmes do período OOS por qualidade de detecção (metodologia de antecedência descrita no texto). Dos 40 alarmes, 4 são falhas confirmadas de comunicação/instrumento (**Comm Fail/Out of Serv** no dado bruto do SCADA), sem precursor físico esperado – não removidos desta contagem para manter a comparação simples, mas documentados em detalhe em `docs/analise_cnn1dae_exp13.md`.

O EXP15b desloca massa de “reativo” para “preditivo” (15→17, e sobretudo reativo caindo de 12 para 8) e reduz “sem detecção” de 10 para 9 – ganho líquido pequeno em contagem bruta, mas o episódio 2026-04-14 (3 alarmes do mesmo dia, antes inteiramente perdido) agora conta como detecção real, e o aumento de “suspeito” (3→6) é o preço de um modelo genuinamente mais sensível: mais casos próximos do teto de 24h da janela de avaliação, consistente com um threshold mais permissivo em valor absoluto de sinal (mesmo mantendo o mesmo critério relativo).

4.3.1 Antecedência de detecção, caso a caso

A Figura 6 detalha a métrica por trás da classificação da Figura 5: para cada um dos 40 alarmes, quantas horas antes (ou depois) do timestamp oficial do alarme a primeira detecção ocorreu, para os dois candidatos lado a lado. Além de permitir auditar caso a caso – por exemplo, confirmar visualmente que os três alarmes de 2026-04-14 têm marcador (triângulo laranja) só para o EXP15b, e nenhum marcador (círculo azul) para a task 14 – o gráfico deixa claro um padrão que a métrica agregada esconde: quando os dois candidatos detectam o *mesmo* alarme, a antecedência é quase sempre muito parecida entre eles (os pares de marcadores tendem a cair próximos um do outro no eixo x) – a recalibração não mudou *quando* um alarme já detectado seria pego, só ampliou o *conjunto* de alarmes detectados.

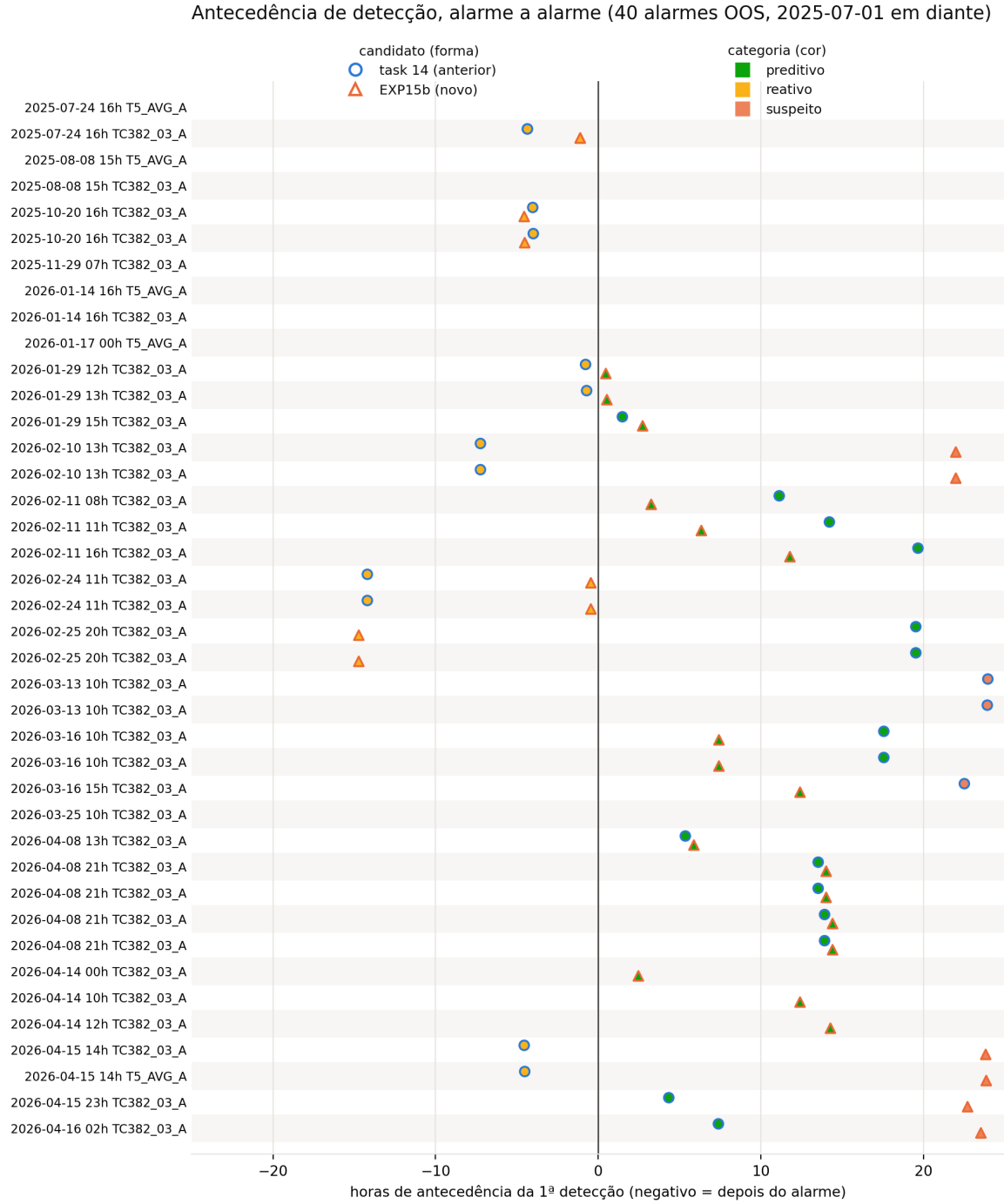


Figura 6: Antecedência da primeira detecção, alarme a alarme (40 alarmes OOS, ordem cronológica). Círculo azul = task 14; triângulo laranja = EXP15b; cor de preenchimento = categoria (Figura 5). Linha vertical em 0 = instante do alarme. Ausência de marcador = alarme sem detecção naquele candidato.

A Figura 7 resume só as detecções preditivas (as que efetivamente anteciparam o alarme) numa distribuição comparável. O EXP15b tem mais detecções preditivas (17 vs. 15), mas com antecedência *menor* em média (8,5h vs. 12,8h; mediana 7,4h vs. 13,9h) – os casos novos que o EXP15b passou a capturar (incluindo os três de 2026-04-14, com antecedência de 4,3h a 7,4h) tendem a ser detecções mais próximas do instante do alarme que a média dos casos que o candidato anterior já capturava, puxando a distribuição toda para antecedências menores.

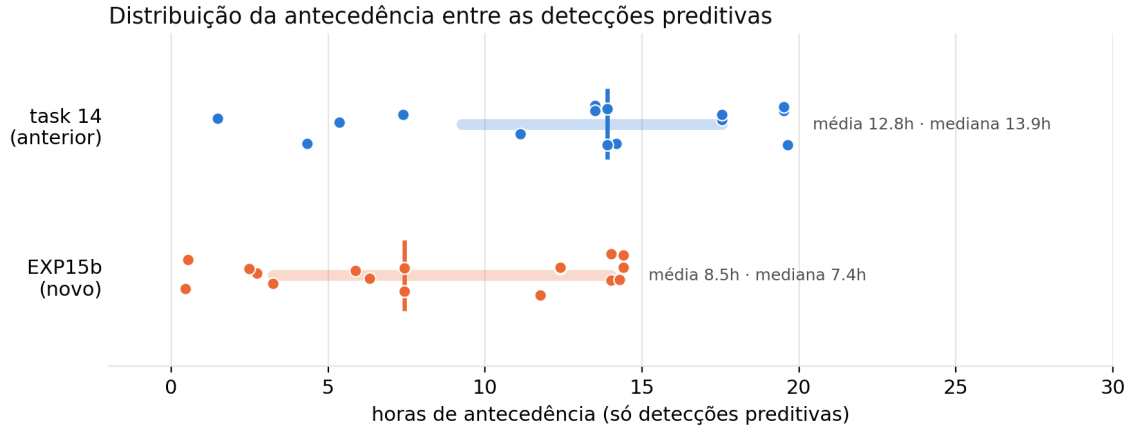


Figura 7: Distribuição da antecedência (horas) entre as detecções classificadas como preditivas, por candidato. Barra horizontal = IQR; traço vertical = mediana.

4.4 Variação entre sementes de retreino (seed-sweep)

A comparação seed-sweep contra seed-sweep (Figura 8) mostra que o EXP15b supera o candidato anterior em **todos** os pontos – semente principal, média, mínimo e máximo do sweep:

Candidato	seed principal	média sweep	std	mín	máx
task 14 (anterior)	75,0%	50,0%	$\pm 7,7\text{pp}$	37,5%	57,5%
EXP15b (novo)	77,5%	60,6%	$\pm 9,9\text{pp}$	47,5%	75,0%

Tabela 3: Seed-sweep ($\text{SEED_SWEEP_N}=4$, sementes 43–46) de cada candidato. O desvio-padrão em pontos percentuais é maior no EXP15b, mas o coeficiente de variação (std/média) é praticamente igual entre os dois (16,3% vs. 15,4%) – o EXP15b não é “mais instável”, só parte de uma base mais alta.

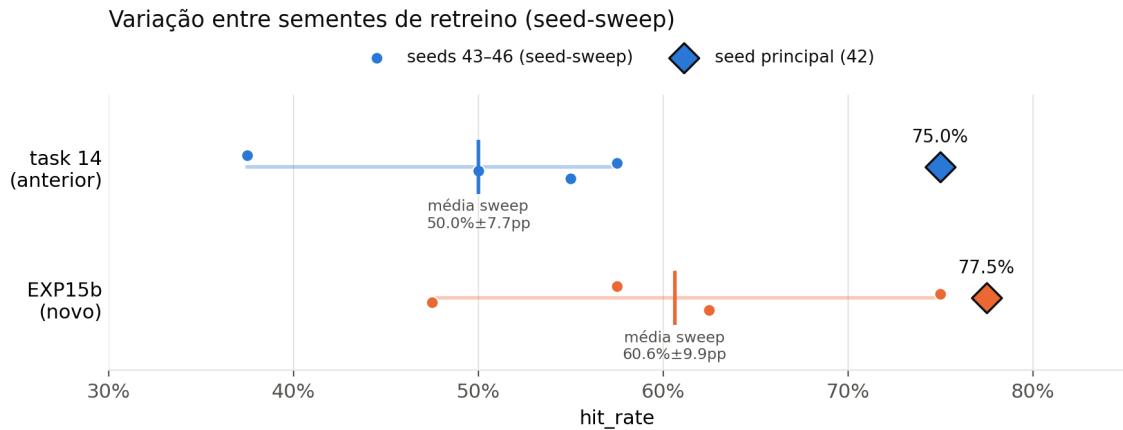


Figura 8: Dispersão do `hit_rate` entre a semente principal (42, losango) e as 4 sementes do sweep (círculos, com a média marcada por um traço vertical), para os dois candidatos.

4.5 Séries completas com anomalias marcadas

As Figuras 9 e 10 mostram a série bruta completa de TC382_03_A de cada candidato, com o estado operacional (faixa inferior: verde=ligada, laranja=desligada) gerada automaticamente pela pipeline – referência visual da amplitude completa da série (incluindo os $\sim 1,9$ anos de dados de treino) e do padrão intermitente de liga/desliga que motiva a máscara operacional (Seção 2.3).

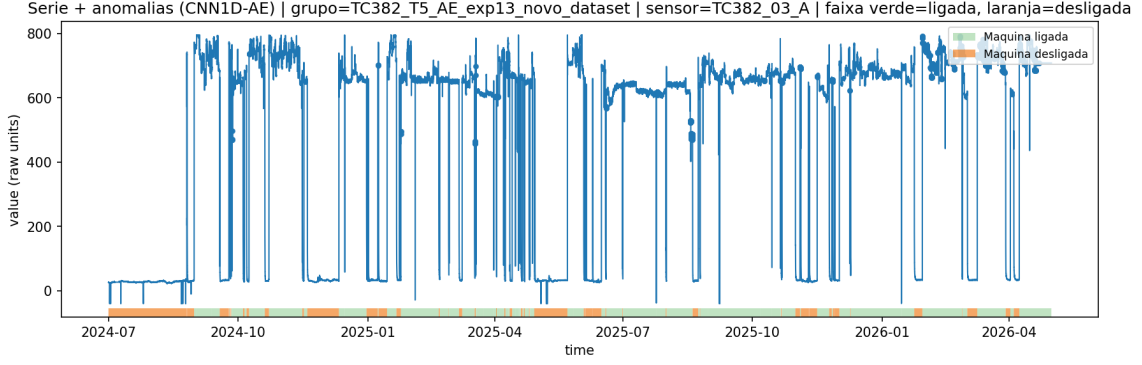


Figura 9: Série completa de TC382_03_A – candidato anterior (task 14).

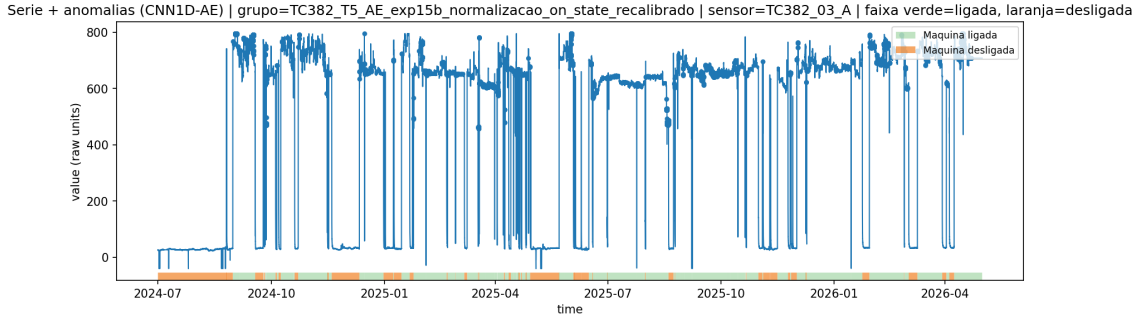


Figura 10: Série completa de TC382_03_A – candidato novo (EXP15b).

5 Discussão

5.1 Tentativa de mitigação de variância rejeitada

Antes de aceitar o EXP15b como candidato final, testamos se `THRESH_MODE=target_rate` (fixar a taxa de anomalia do treino em vez de $\text{mediana} + \text{MAD}$) reduziria a variância de semente sem custo. O resultado foi negativo: o threshold saiu $2,3\times$ mais alto (2,4743 vs. 1,0892), porque o quantil top-0,3% da distribuição de erro de treino é bem mais extremo que $\text{mediana} + K \times \text{MAD}$ nesta distribuição específica. `hit_rate` da semente principal caiu para 15,0% (6/40); a variância de fato caiu (std $\pm 3,2\text{pp}$ vs. $\pm 9,9\text{pp}$), mas porque as 4 sementes do sweep convergiram para um piso de detecção quase inútil (5,0%–12,5%), não porque a calibração melhorou – e o pico de MAE do episódio 2026-04-14 (1,93) ficou abaixo desse threshold mais alto, ou seja, essa variante voltava a perder o episódio que motivou todo o EXP15/EXP15b. `target_rate` fica descartado como modo de threshold para este pipeline nesta taxa-alvo.

5.2 Limitações

- **Efeito colateral da normalização on-state** – (Seção 3.6) os períodos off/partida continuam nas sequências de treino, agora como outliers extremos em z-score. Não investigado se isso degrada a qualidade de reconstrução do autoencoder de forma mensurável além do efeito já observado na escala do threshold.
- **Amostra pequena do seed-sweep** ($N = 4$) – os números de variância da Seção 4.4 têm incerteza estatística considerável; `SEED_SWEEP_N` maior daria uma leitura mais confiável antes de qualquer decisão que dependa fortemente do desvio-padrão exato.
- **Classificação preditivo/reativo/suspeito automatizada** – a Figura 5 usa um critério de antecedência simples (limiar de 20h), mais simplificado que a auditoria manual caso a caso feita

para o candidato anterior no `docs/analise_cnn1dae_exp13.md` (que cruzou cada caso com o dado bruto e o código de status do SCADA). As duas metodologias concordam no número de casos preditivos do candidato anterior (15), mas não foram desenhadas para produzir números idênticos em outras categorias.

- **Gap de cobertura para o AutoML** – mesmo após o EXP15b, o CNN1D-AE (77,5%) ainda fica 15 pp atrás do melhor candidato AutoML (92,5%). O EXP15b fecha parte do gap fechado por task 14 (13,3 pp→4,2 pp de cobertura genuína, conforme `analise_cnn1dae_exp13.md`) mas não o elimina.

5.3 Pendências

- Decidir se o EXP15b substitui definitivamente a task 14 como candidato de produção, ou se o trade-off cobertura×FP justifica manter os dois como opções calibráveis por operador.
- Investigar se filtrar diretamente as linhas de treino pelo estado operacional (em vez de só as estatísticas de normalização) resolve o efeito colateral de outliers extremos sem reintroduzir o problema original de descontinuidade de sequências – por exemplo, treinando com sequências geradas separadamente por trecho contínuo de operação, sem concatenar através de um gap off/partida.
- O episódio de 2026-03-25 (bloqueio de gate por margem pequena, ver `analise_cnn1dae_exp13.md`) segue sem correção – candidato: um segundo `GATE_ESCAPE_MULTIPLIER` mais permissivo, calibrado especificamente para margens de cruzamento pequenas.
- Formalizar a exclusão dos 4 alarmes de erro de sensor confirmado (Comm Fail/Out of Serv) na métrica de avaliação em código, não só em documentação.

6 Conclusão

O EXP15b é o novo candidato de referência do CNN1D-AE, superando o candidato anterior (task 14) em cobertura de alarme (77,5% vs. 75,0%) e em todos os pontos do seed-sweep, à custa de um falso alerta ~37% relativo maior – ainda bem abaixo do teto do AutoML. A causa raiz corrigida (contaminação da normalização por período off/partida) foi identificada através da investigação de um episódio real específico (2026-04-14) e confirmada tanto pela mudança de sinal observada diretamente nos dados (Figura 4) quanto pela melhoria consistente da métrica agregada. Boa parte da metodologia que tornou essa e outras correções possíveis – máscara operacional com segundo critério, portão de rampa refinado, portão de volatilidade – não nasceu no CNN1D-AE: veio de uma investigação equivalente, mais barata de iterar, na pipeline AutoML irmã, documentando o valor de manter as duas pipelines como plataformas de descoberta compartilhada, não só como candidatos concorrentes.

A Tasks ClearML referenciadas

- AutoML EXP10c (referência externa): `6ac3b1b52a45433a83568d61fafadda6`
- CNN1D-AE task 13 (K=7,0, sem gate-escape): `8a95bccb0e40461ea9caea20c94dae10`
- CNN1D-AE task 14 (candidato anterior, + gate-escape): `f8b884932a2441b987086b611182fe1d`
- CNN1D-AE EXP15 (rodada 1, K=7,0 herdado – resultado misto): `39d73f7cb7ae4ce7845903246edd5df9`
- CNN1D-AE EXP15b (candidato novo, K=4,5 recalibrado): `a273ea8c9f674e8ba04ac291f45d2795`
- CNN1D-AE EXP15c (`target_rate`, rejeitado): `b66bbaa24f7e42259d9eec7a90775e48`